

EECS 348 Assignment 1 — GenAI Code Analysis

Name: Peter Lauman

Course: EECS 348 – Software Engineering I

Assignment: Assignment 2

Date: September 17, 2026

1. GenAI Programs Used

The two Generative Artificial Intelligence programs I used for this assignment were Claude and Google Gemini.

2. Collaboration and Sources

I did not collaborate with any other students on this assignment. I used Claude and Google Gemini as the two required GenAI programs. I also used ChatGPT to assist with comparing the generated programs, identifying possible improvements, testing edge cases, improving comments, and organizing the analysis. I reviewed, tested, and implemented the final changes to the program.

3. How the GenAI Programs Were Accessed

I accessed Claude through the Claude website using a Google web browser. I accessed Google Gemini through the Gemini website using a Google web browser.

4. Prompt Used

I used the exact same prompt for Claude and Google Gemini as required by the assignment.

Write a complete C program that prioritizes emails for a busy company CEO using a priority queue implemented as a MaxHeap.

Requirements:

1. You must implement the MaxHeap from scratch using a list-based implementation. Do not use any pre-existing heap modules or libraries.
2. Each email contains:
 - Sender category
 - Subject line
 - Date
3. Emails are prioritized by sender category in this exact order, from highest to lowest priority:
 - Boss
 - Subordinate
 - Peer
 - ImportantPerson
 - OtherPerson
4. If there is more than one email from the same sender category, the newest email must have higher priority than the older email.
5. Dates are provided in MM-DD-YYYY format.
6. The program must read commands from a test file. The possible commands are:

EMAIL <sender category>,<subject line>,<date>

Adds the email to the MaxHeap according to its priority.

NEXT

Displays the highest-priority unread email in exactly this format:

Next email:

Sender: <sender category>

Subject: <subject line>

Date: <date>

NEXT must not remove the email from the queue. Therefore, two NEXT commands in a row without a READ should display the same email.

READ

Removes the highest-priority unread email from the MaxHeap. It should not display the email. Two READ commands in a row should remove the two highest-priority emails.

COUNT

Displays the current number of unread emails in exactly this format:

There are <number> emails to read.

7. The program must safely handle NEXT or READ when there are no emails.
8. Assume:
 - Sender category will always be one of the five valid categories.
 - Subject lines may contain spaces but will not contain commas.
 - Dates will always be formatted MM-DD-YYYY.
 - Commands will be correctly formatted, but their order and number may vary.
9. Use standard C, not C++.
10. Write clear, readable, and maintainable code.
11. Include comments explaining the important sections of the program.

Provide the complete source code.

5. Claude Generated Code

Generated code is in the github with this file dude to file size

6. Gemini Generated Code

Generated code is in the github with this file dude to file size

7. Analysis of the Generated Programs

Correctness

Both Claude's and Gemini's programs implemented the main requirements of the assignment. Both programs created a MaxHeap using a dynamically allocated array and prioritized emails according to sender category. Boss emails received the highest priority followed by Subordinate Peer ImportantPerson and OtherPerson. Both programs also compared dates when two emails had the same sender category so that the newest email was given the higher priority.

Both programs correctly implemented the basic behavior of next, read, and count. next accesses the email at the root of the MaxHeap without removing it while read removes the highest priority email. count displays the current number of unread emails. This matches the required behavior in the assignment.

Claude's program included additional processing functions and checked whether its calls to malloc and realloc succeeded. This made Claude's version more robust if a memory allocation error occurred.

Gemini's program used an enum to represent the sender priorities and stored the parsed year, month, and day in each Email structure. This made the priority comparison easy to understand. However, Gemini did not check whether its calls to malloc and realloc were successful.

During improvement and testing an issue was identified in the insertion sequence inherited from the generated implementations. The final version was changed so that the heap size is increased before heapifyUp is called and the new element is passed using `heap -> size - 1`. This makes the insertion operation clearer and ensures the newly added element is treated as part of the current heap during reorganization.

Execution Time

Both programs have similar execution-time complexity because they use the same basic MaxHeap data structure. Adding an email with EMAIL requires inserting the email and potentially moving it upward through the heap.

The NEXT command accesses the root of the MaxHeap without searching through the other emails so it takes $O(1)$ time.

The READ command removes the root and may need to move the replacement email downward through the heap. Therefore READ takes $O(\log n)$ time in the worst case.

The COUNT command only accesses the stored heap size so it takes $O(1)$ time. The additional memory checks and helper functions in Claude's implementation do not change its overall time complexity.

Space Complexity

Both programs have $O(n)$ space complexity because the MaxHeap stores each unread email in a dynamically allocated array. As more emails are added the amount of memory needed increases with the number of emails.

Claude stores the category, subject, and date strings for each email. Gemini stores those values and also stores the parsed year, month, and day. Gemini uses slightly more memory for each email, but both programs still have the same overall $O(n)$ space complexity.

The temporary variables used by heap operations require only a fixed amount of additional memory so they do not change the overall space complexity.

Maintainability

Both programs divide the MaxHeap operations into functions, which makes the programs easier to understand than placing all heap operations directly in main. Claude's program is more modular because it separates tasks such as input parsing, whitespace removal, command handling, heap operations, and priority comparison into individual functions. This makes individual parts of the program easier to modify or debug. Gemini's use of an enum for sender categories is easy to understand because the numerical priorities are directly associated with named categories. However, more of Gemini's command-processing logic is located directly inside main. Claude's code also checks for memory allocation failures which makes the program more robust and easier to troubleshoot. For these reasons Claude's program provided a better starting point for the final implementation.

8. Program Selected

I selected Claude's program as the base for my final program. Both Claude and Gemini correctly implemented the required MaxHeap and had similar execution time and space complexity. However, Claude's implementation was more modular because it separated heap operations, input parsing, and command handling into individual functions. Claude's program also checked whether dynamic memory allocation was successful. Gemini's implementation did not perform these checks. This made Claude's program more robust and provided a better starting point for improvements. Gemini still provided useful ideas for the analysis. In particular its use of clear priority values and separate date components showed another simple way of comparing email priorities.

9. Improvements Made to Claude's Program

Correctness

I improved the correctness of Claude's program by testing the MaxHeap operations and making the insert sequence clearer. The final version places the new email at the end of the array, increases the heap size, and then calls `heapifyUp` using `heap -> size - 1` as the index of the newly inserted email. I also made sure that `next` does not remove an email from the MaxHeap and that `read` removes an email without displaying it. This allows two consecutive `next` commands to display the same email and two consecutive `read` commands to remove the two highest priority emails as required by the

assignment. The final program also safely handles next and read when the heap is empty. These are specifically identified as possible grader test cases. The program was tested using the sample test file provided in the assignment. The expected sample begins with six emails and should display the newer Boss email first followed by the older Boss email after the first one is read.

Execution Time

The improvements do not change the overall execution-time complexity of Claude's program. email insertion and read remain $O(\log n)$ in the worst case because they may require moving through the height of the MaxHeap. next and count remain $O(1)$ because they directly access the root or stored heap size. The additional error checking and comments do not significantly affect program execution time.

Space Complexity

The improved program continues to have $O(n)$ space complexity because the dynamically allocated MaxHeap grows according to the number of unread emails. The final version stores only the sender category, subject, and date for each email. The additional variables used during comparisons and heap operations require only a fixed amount of memory and therefore do not change the overall space complexity.

Maintainability

I improved maintainability by adding a complete program prologue, descriptive constants, comments explaining the major sections of the program, and comments explaining the individual operations.

I also identified the sources used in the program and indicated which sections were based on Claude's generated implementation and which were modified with assistance from ChatGPT. This follows the assignment requirement that the program identify collaborators and other sources of code.

I kept separate functions for category priority, date conversion, email comparison, heap initialization, heap growth, insertion, removal, and heap reorganization. This makes the final program easier to read, test, debug, and modify.